Here is a complete, step-by-step breakdown of every advice, tip, and potential time-saving bug prevention strategy outlined in the PDF, ordered strictly by the implementation sequence of the task:

Step 1: Data Preparation & Constants

- **Data Reshaping & Data Type:** Convert images into 1D vectors and cast to float64 explicitly. Do not use float32.
- **Data Layout (Examples in Columns):** Input X must have shape $(784, m)$ and Y must have shape $(10, m)$, where m is the batch size.
- **Target Format:** In NumPy, Y must be a one-hot encoded matrix.

Step 2: Weight Initialization (initialize_parameters)

- **Avoid Zero Initialization:** Initializing all weights to zero causes the symmetry problem. Every neuron computes identical values, gets identical gradients, and updates identically—effectively reducing a layer of 128 neurons into 1 neuron. Biases can be zero because random weights break the symmetry.
- **Avoid Large Random Weights (e.g., * 1.0):** Summing hundreds of terms leads to large pre-activations (Z), causing exponential activation explosion, Softmax overflow, and NaN values.
- **Avoid Very Small Random Weights (e.g., * 0.01):** Shrinks the signal rapidly across layers, leading to tiny gradients and stalled learning.
- **Use He Initialization Correctly:** Draw weights using $W^{[l]} \sim \mathcal{N}(0, 2/n_{l-1})$. The factor of 2 compensates for ReLU zeroing out half the inputs and destroying half the variance.
- **Maintain Strict Random Generator Ordering:** Generate $W^{[l]}$ then $b^{[l]}$ in sequential order from layer 1 to layer L inside the loop. Swapping the order or drawing $b^{[l]}$ randomly advances the NumPy random seed differently, resulting in non-reproducible weights.

Step 3: Forward Propagation (layer_forward & model_forward)

- **Store Caches for the Backward Pass:** Always return both activation $A^{[l]}$ and pre-activation $Z^{[l]}$ during forward passes. Recomputing Z in backprop is redundant and wasteful.
- **Softmax Numerical Stability:** Standard e^{z} calculation overflows when $z > 700$, producing inf and NaN. Subtract the column-wise maximum ($\max_j z_j$) prior to exponentiating.
- **Keep Dimensions during Softmax Subtraction:** Always set axis=0 and keepdims=True during column-wise max subtraction so broadcasting occurs correctly per example.

Step 4: Cost Computation (compute_cost)

- **Avoid $\log(0)$ Undefined Errors:** Add a small constant ϵ (e.g., 1e-12) inside the logarithm or clip the predicted probabilities $A^{[L]}$ to prevent computing $\log(0)$, which outputs $-\infty$ and ruins downstream computations.

Step 5: Backward Propagation (layer_backward & model_backward)

- **The Single Most Common Bug ($1/m$ Scaling):** Ensure $dW^{[l]}$ and $db^{[l]}$ are

explicitly averaged over the batch size m using the $\frac{1}{m}$ factor. Do **NOT** apply $1/m$ to $dA^{[l-1]}$. Forgetting this makes gradients $64\times$ too large; the model will still appear to train reasonably, but gradient checks and numerical precision tests will fail.

- **Derive Transposes via Shape Assertions:** Prevent shape mismatch errors by checking dimensions:
  - $dW^{[l]}$ shape must match $W^{[l]} \rightarrow (n_l, n_{l-1})$.
  - $dA^{[l-1]}$ shape must match $A^{[l-1]} \rightarrow (n_{l-1}, m)$.
  - Always use assert statements after every shape computation to catch unintended broadcasting immediately.
- **Correct Summation for Bias Gradients:** Sum $dZ^{[l]}$ across columns (axis=1) and maintain keepdims=True to preserve shape $(n_l, 1)$ instead of flattening into 1D.

Step 6: Mini-Batch Training & Model Class

- **Joint Dataset Shuffling:** When shuffling inputs prior to splitting into mini-batches, apply the exact same permutation array to both X and Y. If X and Y decouple, image-label pairing is lost and the network cannot learn.
- **Handle Dynamic Last Batch:** The final mini-batch in an epoch might have fewer than 64 samples; Ensure m in gradient calculations uses the dynamic batch size (A_prev.shape[1]) rather than a hardcoded fixed 64.

Step 7: PyTorch Implementation (Part B)

- **Do NOT Apply Softmax in forward():** PyTorch's nn.CrossEntropyLoss automatically combines Softmax and Cross-Entropy into a numerically stable single function. Applying Softmax inside the model's forward pass applies Softmax twice silently, degrading model performance without throwing an error.
- **Label Representation:** PyTorch requires target labels Y as a 1D tensor of class integer indices of shape $(m,)$, not one-hot vectors.
- **Data Layout (Examples in Rows):** PyTorch expects input shape $(m, 784)$, unlike NumPy's $(784, m)$.
- **Clear Accumulated Gradients:** Call optimizer.zero_grad() before loss.backward() during every iteration. PyTorch accumulates (adds) gradients by default, causing incorrect gradient accumulation across steps if omitted.